Gatekeeper: Full Build Prompt

Copy everything below into your coding model as the starting prompt.

Role and goal

You are a senior engineer helping me build an RL environment for a 48-hour hackathon titled "Build an RL Environment That Can't Be Broken". I build the environment, harness and verifier. The judges bring the agent: an LLM called through an OpenAI-compatible chat API. I do not train or build any agent.

The project is **Gatekeeper**, an access-policy repair environment. Work step by step, write tests as you go, and keep dependencies minimal and pinned.

The idea

A fictional company has users, groups and resources. Its access policy (allow and deny rules) is broken: some people can do things they should not (leaks) and some cannot do things they should (outages). The agent receives plain-English requirements plus the broken policy. It investigates with tools, edits the policy, and calls `submit`. A hidden verifier then compares the final policy with the correct one across tens of thousands of requests the agent never saw.

Key design principle: **generate the correct policy first, then break it.** This makes the reference solution trivial (undo the breakage), makes any seed solvable, and makes scoring objective.

Hackathon rules the build must satisfy

Three bars: it works, it is model-agnostic, it is robust. Robustness (Run B, 40 of 100 points) carries the most weight.

Minimum spec (all required):

- Task definition with explicit success criteria.
- Observation shown at each step.
- Action/tool space where every tool has a name, description and argument schema.
- State that changes with actions and persists within an episode.
- `reset(seed)` giving the same starting state and the same state hash for the same seed.

- Termination on success, failure, step cap, time cap or crash, with the reason recorded.
- Verifier and reward using programmatic checks only (no LLM-as-judge). Partial credit is encouraged.
- At least 5 tasks across 2 or more difficulty levels (more or generated tasks score higher).
- One named demo task that GPT-Luna can realistically finish within the live-run limits.
- Reference solution and no-op baseline for every task.
- Isolation: the agent cannot reach the verifier, hidden data or reward code, and has no network except the model API.
- One command from a fresh clone: `docker compose up --build --abort-on-container-exit --exit-code-from harness`.

Baseline gate (runs before the model is ever called):

- Reference solution must score at least 0.95.
- No-op agent (submits immediately) must score at most 0.05.
- If either fails, the run stops and counts as failed. A no-op above 0.05 means undeserved reward, which is the most serious failure.

Live-run limits: build 3 minutes (empty Docker cache); baseline checks 2 minutes together; model run 10 minutes wall-clock from the first model request; 40 agent steps (one step is one model reply, regardless of tool calls); at most 10 tool calls per reply (extra calls are not run and return an error); 60 seconds per tool call (timeout returns an error and the episode continues); verifier scoring within 2 minutes; whole slot 16 minutes hard stop. Plan for a 64K-token context: cap tool output and trim old observations. Overflowing the context counts as a crash.

Harness contract:

- Inputs from `.env`: `MODEL`, `API_KEY`, `BASE_URL`, `TASK_ID`, optional `SEED`, `APPEND_FILE` (default `inputs/append.txt`), `MODE` (`full`, `all`, `baselines`).
- The contents of `APPEND_FILE` are added to the end of the system prompt exactly as given and logged. If the file is explicitly named and is missing or empty, the run stops. If it is not named, add nothing.
- All models are called through the same OpenAI-compatible interface (`openai` Python client, `chat.completions.create` with tools). The harness must not special-case any model or the judges' instructions.
- Robust parsing of whatever the model returns, well-formed or not. An unreadable reply still uses a step; send the agent a clear error and carry on.
- Pin every dependency to an exact version including transitive ones, install only what is needed, and download nothing at run time.

- No API keys in the repo, zip or logs. Ship `.env.example` only.
- All environment code must be written during the hackathon (template repo as a starting point is allowed).

World design

- About 30 to 120 users in groups (with optional nesting on hard tiers), 40 to 200 resources tagged with department and sensitivity, actions `read`, `write`, `delete`, `share`, and request context fields such as `network` and `mfa`.
- **Policy statements:** `id`, `effect` (allow or deny), `principals` (users or groups), `actions`, `resources` (ids, tags or wildcards), optional `condition` (context equals value), optional exceptions.
- **Evaluation semantics (document in the system prompt):** default deny; an explicit deny beats any allow; conditions must all hold for a statement to apply.
- `decide(policy, user, action, resource, context)` returns allow or deny plus the id of the deciding statement.
- Use "access policy" or "permission ruleset" in docs, never just "policy", to avoid confusion with the RL notion of an agent policy.

Task generation

Pipeline: `seed -> world -> correct policy P* -> English requirements rendered from P* -> k mutations -> broken policy`.

- Use one `random.Random(seed)` for everything so the same seed gives identical files and an identical state hash.
- Mutation catalog (4 to 6 types): drop a statement, widen a wildcard, remove a condition, remove a deny, change a group, flip an effect. Record each mutation in a hidden answer key.
- Resampling rule: resample until each mutation changes a meaningful number of decisions, and until allow-all and deny-all would both score 0.
- Requirements must pin down the correct behavior so that the intended fix is inferable. Any equivalent policy that gives the same decisions must score the same.
- Tiers: easy has 1 to 2 mutations and about 8 requirements; medium has 3 to 4 mutations with exceptions and conditions interacting; hard has 5 to 8 mutations with nested groups and overlapping wildcards.
- `tasks.json` defines 12 or more named tasks, for example `{"task_id":"med_03","tier":"medium","seed":42,"n_users":60,"n_resources":100,"mutations":3,"mutation_types":[...]}`. `TASK_ID` selects the config; optional

`SEED` overrides the seed.

- Demo task: a medium task a capable model can finish within 40 steps, named in the README.

Agent tools (action space)

Each tool has a name, description and pydantic argument schema, and returns compact, capped output:

- `get_requirements()`
- `list_policies(page)` (paginated)
- `lookup(kind, id)` for a user, group or resource
- `simulate(principal, action, resource, context)` returns the current policy's decision and the deciding statement, never the truth
- `put_policy(statement)`, `delete_policy(id)`, `undo()`, `diff()`
- `check_examples()` returns a small fixed visible sample
- `submit()` ends the episode

Observations are short one-line summaries. Unknown tools, bad arguments and extra calls return clear errors and the episode continues.

Verifier and reward

- Runs in a protected service the agent cannot reach. At `submit`, it takes the final policy snapshot and evaluates the full hidden decision matrix (all user, action, resource and context combinations) for both the final policy and P*.
- Errors: a false allow (leak) weighs 3, a false deny (outage) weighs 1. `score = max(0, 1 - E_final / E_start)`.
- Expected scores: oracle 1.0; no-op 0; allow-all 0; deny-all 0; hard-coding the visible examples about 0.
- No feedback loop: nothing from the hidden matrix reaches the agent. Log a per-step shadow reward for telemetry only.
- Optimize for speed (precompute group memberships and indexes) so scoring finishes well inside 2 minutes.
- Differential test: check the evaluator against an independent naive implementation on thousands of random policies.

Baselines

- Oracle: reverses the recorded mutations for any seed, must score at least 0.95.
- No-op: submits immediately, must score at most 0.05.
- A script runs both on every task (`MODE=baselines`) and writes `gate.json`.

Architecture and isolation

One pinned image, three services in Docker Compose:

- `harness`: model client, loop, parser, limits, logs. The only service with access to the model API.
- `env`: world state and tools. No API key, no egress.
- `verifier`: generator, answer key, scoring. No egress, unreachable from the agent's workspace.

Hardening: internal Docker network, non-root users, read-only filesystems, memory and pid limits, healthchecks (because `--abort-on-container-exit` stops everything if any sidecar dies).

Harness details

- Tolerant tool-call parser (handles missing fields, invalid JSON, text-only replies, multiple calls).
- Cap 10 tool calls per reply; run them in order and return results together.
- Cap tool output size; trim or summarize old observations to stay within 64K tokens. Estimate tokens with characters divided by 3 plus API `usage` counts (do not use `tiktoken`, which downloads data at run time).
- Enforce 40 steps, 10 minutes and the 60-second tool timeout.
- Termination reasons recorded: `success`, `failure`, `step_cap`, `time_cap`, `crash`.

Logging and telemetry

Write JSONL to `./logs/<run>/episode.jsonl`, one line per model reply, with: `timestamp`, `step`, `seed`, `model_input`, `model_output_raw`, `parsed_actions` (or parse error), `tool_result`, `errors`, `state` (hash or snapshot), `reward` (components and total), `tokens`, `latency_ms`. Also record the verifier's final output and the termination reason, and log the baseline checks the same way. Print a readable stream to the terminal.

Flag suspicious behavior: unknown tool names, path-like arguments, edits that allow everything, repeated identical calls, `simulate` sweeps, context above 80 percent, parse-failure streaks.

Cheater test suite

Scripted agents with asserted outcomes: allow-all, deny-all, hard-code the visible examples, `simulate` sweep, path probe, giant payload, 25 calls in one reply, malformed JSON, submit spam. Record the scoreboard in the design doc.

Libraries

`openai`, `pydantic`, `fastapi` and `uvicorn` (or standard library HTTP), `pytest`, optional `hypothesis`, and `uv` or `pip-tools` for a fully pinned lockfile. Avoid pandas and `tiktoken`.

Repo layout

```
repo/
  docker-compose.yml  .env.example  inputs/append.txt
  harness/  env/  verifier/  generator/  baselines/
  tasks.json  tests/  README.md
```

Build order

1. Evaluator with unit tests and the differential test.
2. Generator with determinism and state-hash tests.
3. Mutations and the hidden answer key.
4. Verifier.
5. Oracle and no-op baselines passing the gate on all tasks.
6. Env tools with schemas.
7. Harness: parser, limits, trimming, logs, `APPEND_FILE` handling.
8. Docker Compose with three services and isolation.
9. More mutation types and tasks (12 or more), then test with two different model styles and a fake model server.
10. Cheater suite, design doc (1 to 2 pages: task, reward design, known weaknesses, defences) and deck.

If the evaluator, generator and baselines are not green over 1000 seeds by about hour 14, switch to a simpler idea.

Submission deliverables

Private GitHub repo named exactly after the team (main branch is what gets run), plus `TeamName.zip` containing `baseline-results/` (`gate.json` covering all tasks), `test-run-logs/` (at least 3 model runs), `sample-trajectories/` (at least one success and one failure), `design-doc.pdf` and `deck.pdf`.

How I want you to work

Start with step 1. Show the code, the tests and how to run them. Ask me only when a decision truly blocks progress, and state your assumptions otherwise.